Module 4 : Commodity Data Sources and Quality

Learning Objectives

By the end of this module, you will:

1. Understand the landscape of commodity data sources (free and premium)
2. Access data programmatically via APIs (yfinance, FRED, EIA)
3. Assess data quality across multiple dimensions
4. Handle common data issues while maintaining temporal discipline
5. Build a robust data pipeline for commodity research

Why Data Quality Matters

"Garbage in, garbage out" is especially true in commodity trading:

- Bad data leads to false signals
- Missing data creates look-ahead bias
- Delayed data makes backtests unrealistic
- Inconsistent data breaks models in production

This module teaches you to source, validate, and clean data properly.

In [1]:
```
# Setup and imports
import sys
sys.path.append('../..')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

# Fair Value Toolkit imports
from src.fair_value_toolkit import (
    PointInTimeDataFrame,
)

from data.data_fetchers import (
    CommodityDataFetcher,
    COMMODITY_TICKERS,
    FRED_SERIES,
    create_sample_dataset,
    prepare_training_data,
)

# Plotting configuration
plt.style.use('seaborn-v0_8-darkgrid')
```

```
sns.set_palette("husl")
plt.rcParams['figure.figsize'] = (14, 6)
plt.rcParams['font.size'] = 10

print("Environment ready!")
```

```
Environment ready!
```

Part 1 : Data Source Overview

1.1 Free Data Sources

Yahoo Finance

- **Coverage**: Futures prices for major commodities
- **Frequency**: Daily (historical), real-time (delayed)
- **History**: Typically 5-10 years
- **Cost**: Free
- **Reliability**: Good for price data, limited fundamentals

FRED (Federal Reserve Economic Data)

- **Coverage**: Economic indicators, some commodity spot prices, inventories
- **Frequency**: Various (daily, weekly, monthly)
- **History**: Extensive (decades for many series)
- **Cost**: Free
- **Reliability**: Excellent, official government data

EIA (Energy Information Administration)

- **Coverage**: Energy commodities (oil, gas, coal, electricity)
- **Frequency**: Various (weekly inventories, monthly production)
- **History**: Extensive
- **Cost**: Free
- **Reliability**: Excellent, official government data

In [2]:
```
# Display available commodity tickers
print("Available Commodity Tickers (Yahoo Finance):\n")
for category in ['Energy', 'Precious Metals', 'Base Metals', 'Agriculture
    print(f"\n{category}:")

tickers_df = pd.DataFrame.from_dict(COMMODITY_TICKERS, orient='index', co
print(tickers_df.to_string())
```

```
Available Commodity Tickers (Yahoo Finance):


Energy:

Precious Metals:

Base Metals:

Agriculture:
             Ticker
crude_oil      CL=F
brent_oil      BZ=F
natural_gas    NG=F
gasoline       RB=F
heating_oil    HO=F
gold           GC=F
silver         SI=F
platinum       PL=F
palladium      PA=F
copper         HG=F
corn           ZC=F
wheat          ZW=F
soybeans       ZS=F
soybean_oil    ZL=F
soybean_meal   ZM=F
coffee         KC=F
sugar          SB=F
cotton         CT=F
cocoa          CC=F
live_cattle    LE=F
lean_hogs      HE=F
```

In [3]:

```python
# Display available FRED series
print("Available FRED Series:\n")
fred_df = pd.DataFrame.from_dict(FRED_SERIES, orient='index', columns=['S
print(fred_df.to_string())
```

```
Available FRED Series:

                        Series Code
fed_funds                   FEDFUNDS
treasury_3m                     DTB3
treasury_10y                   DGS10
dollar_index               DTWEXBGS
cpi                        CPIAUCSL
pce                           PCEPI
industrial_production        INDPRO
unemployment                 UNRATE
wti_spot                  DCOILWTICO
brent_spot              DCOILBRENTEU
gas_price                   GASREGW
crude_inventory            WCESTUS1
natural_gas_inventory      NGINSTUS
```

1.2 Premium Data Sources (Overview)

While we'll focus on free sources in this course, be aware of premium options:

Bloomberg Terminal

- **Coverage**: Comprehensive (prices, fundamentals, news, analytics)
- **Cost**: ~$ 2 4 , 0 0 0 /year per user
- **Best for**: Professional traders, institutional research

Refinitiv (formerly Thomson Reuters)

- **Coverage**: Similar to Bloomberg
- **Cost**: Similar to Bloomberg
- **Best for**: Institutional users

Quandl (now part of Nasdaq)

- **Coverage**: Curated alternative data, commodities
- **Cost**: Varies ($ 5 0 -$ 1 0 0 0 +/month)
- **Best for**: Quant researchers, algo traders

CME Group DataMine

- **Coverage**: Detailed futures/options data
- **Cost**: Varies by dataset
- **Best for**: Futures-focused strategies

IEA, OPEC, USDA Reports

- **Coverage**: Industry-specific fundamentals
- **Cost**: Free to moderate
- **Best for**: Fundamental analysis

1 . 3 Data Characteristics by Source

In [4]:
```
# Create comparison table
comparison = pd.DataFrame({
    'Source': ['Yahoo Finance', 'FRED', 'EIA', 'Bloomberg', 'Quandl'],
    'Price Data': ['Excellent', 'Limited', 'Good (Energy)', 'Excellent',
    'Fundamentals': ['Poor', 'Good', 'Excellent (Energy)', 'Excellent', '
    'Historical Depth': ['5-10 years', '20+ years', '10+ years', '30+ yea
    'Update Frequency': ['Daily', 'Varies', 'Weekly/Monthly', 'Real-time'
    'API Access': ['Yes (free)', 'Yes (free)', 'Yes (free)', 'Yes (paid)'
    'Cost': ['Free', 'Free', 'Free', '$$$', '$$'],
    'Data Quality': ['Good', 'Excellent', 'Excellent', 'Excellent', 'Good
})

print("Data Source Comparison:\n")
print(comparison.to_string(index=False))
```

```
Data Source Comparison:

       Source    Price Data       Fundamentals Historical Depth Update Freque
API Access Cost   Data Quality
Yahoo Finance     Excellent               Poor       5-10 years            Da
Yes (free) Free           Good
         FRED       Limited               Good        20+ years           Var
Yes (free) Free      Excellent
          EIA Good (Energy) Excellent (Energy)        10+ years   Weekly/Mont
Yes (free) Free      Excellent
    Bloomberg     Excellent          Excellent        30+ years        Real-t
Yes (paid)  $$$      Excellent
       Quandl     Excellent               Good           Varies           Var
Yes (paid)   $$ Good-Excellent
```

Part 2 : API Integration

2.1 Using yfinance for Futures Prices

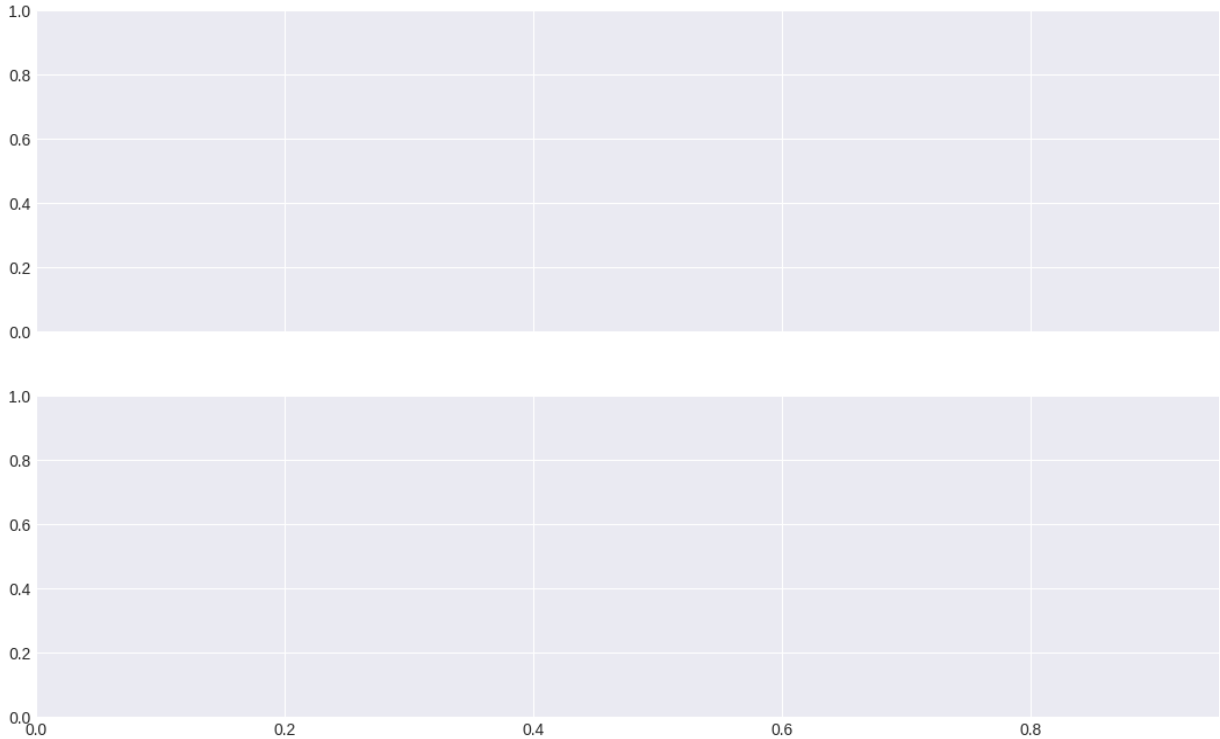
In [5]:
```python
# Initialize data fetcher
fetcher = CommodityDataFetcher(cache_dir='../../data/cache')

# Fetch crude oil futures data
print("Fetching WTI Crude Oil futures data...")
crude_data = fetcher.fetch_yahoo_data(
    ticker='CL=F',
    start_date='2020-01-01',
    end_date='2023-12-31'
)

print(f"\nData shape: {crude_data.shape}")
print(f"Columns: {list(crude_data.columns)}")
print(f"Date range: {crude_data.index.min().date()} to {crude_data.index.
crude_data.head()
```

```
Fetching WTI Crude Oil futures data...
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[5], line 6
      4 # Fetch crude oil futures data
      5 print("Fetching WTI Crude Oil futures data...")
----> 6 crude_data = fetcher.fetch_yahoo_data(
      7     ticker='CL=F',
      8     start_date='2020-01-01',
      9     end_date='2023-12-31'
     10 )
     12 print(f"\nData shape: {crude_data.shape}")
     13 print(f"Columns: {list(crude_data.columns)}")

AttributeError: 'CommodityDataFetcher' object has no attribute 'fetch_yahoo_d
```

In [6]:
```python
# Visualize price and volume
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)

# Price
crude_data['Close'].plot(ax=axes[0], linewidth=1.5, color='blue')
axes[0].set_ylabel('Price ($/barrel)')
axes[0].set_title('WTI Crude Oil Futures - Continuous Contract')
axes[0].grid(True, alpha=0.3)
```

```
# Volume
crude_data['Volume'].plot(ax=axes[1], linewidth=1, color='green', alpha=0
axes[1].set_ylabel('Volume')
axes[1].set_xlabel('Date')
axes[1].set_title('Trading Volume')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[6], line 5
      2 fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)
      4 # Price
----> 5 crude_data['Close'].plot(ax=axes[0], linewidth=1.5, color='blue')
      6 axes[0].set_ylabel('Price ($/barrel)')
      7 axes[0].set_title('WTI Crude Oil Futures - Continuous Contract')

NameError: name 'crude_data' is not defined
```



2.2 Using FRED API for Economic Data

In [7]:
```
# Fetch crude oil inventory data from FRED
print("Fetching crude oil inventory data from FRED...")
inventory_data = fetcher.fetch_fred_data(
    series_id='WCESTUS1',  # Weekly crude oil stocks
    start_date='2020-01-01',
    end_date='2023-12-31'
)

print(f"\nData shape: {inventory_data.shape}")
print(f"Date range: {inventory_data.index.min().date()} to {inventory_dat
inventory_data.head()
```

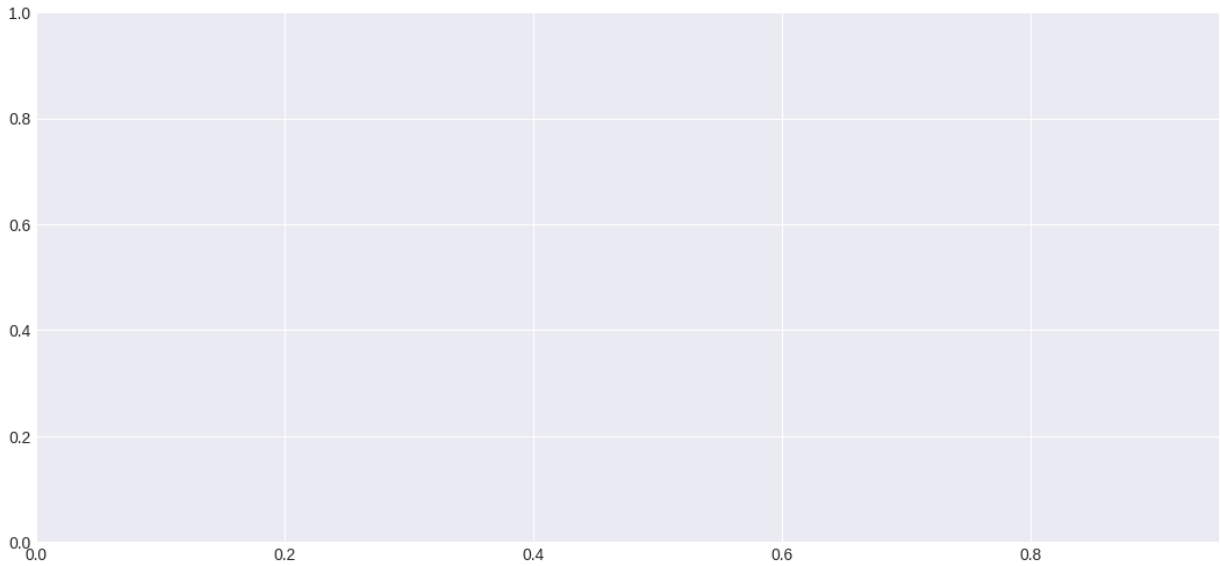
```
Fetching crude oil inventory data from FRED...
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[7], line 3
      1 # Fetch crude oil inventory data from FRED
      2 print("Fetching crude oil inventory data from FRED...")
----> 3 inventory_data = fetcher.fetch_fred_data(
      4     series_id='WCESTUS1',  # Weekly crude oil stocks
      5     start_date='2020-01-01',
      6     end_date='2023-12-31'
      7 )
      9 print(f"\nData shape: {inventory_data.shape}")
     10 print(f"Date range: {inventory_data.index.min().date()} to {inventory_
index.max().date()}")

AttributeError: 'CommodityDataFetcher' object has no attribute 'fetch_fred_da
```

In [8]:

```python
# Visualize inventory levels
fig, ax = plt.subplots(figsize=(14, 6))

inventory_data.plot(ax=ax, linewidth=1.5, color='brown')

# Add 5-year average band
rolling_mean = inventory_data.rolling(window=52*5, center=True).mean()
rolling_std = inventory_data.rolling(window=52*5, center=True).std()

rolling_mean.plot(ax=ax, linewidth=2, linestyle='--', color='black', labe
ax.fill_between(
    inventory_data.index,
    (rolling_mean - rolling_std).values.flatten(),
    (rolling_mean + rolling_std).values.flatten(),
    alpha=0.2,
    color='gray',
    label='±1 std dev'
)

ax.set_ylabel('Inventory (million barrels)')
ax.set_xlabel('Date')
ax.set_title('U.S. Crude Oil Inventory (Weekly)')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[8], line 4
      1 # Visualize inventory levels
      2 fig, ax = plt.subplots(figsize=(14, 6))
----> 4 inventory_data.plot(ax=ax, linewidth=1.5, color='brown')
      6 # Add 5-year average band
      7 rolling_mean = inventory_data.rolling(window=52*5, center=True).mean(

NameError: name 'inventory_data' is not defined
```



2.3 Fetching Multiple Series

In [9]:
```
# Fetch multiple commodities
commodities = ['crude_oil', 'natural_gas', 'gold', 'copper']
multi_commodity_data = {}

print("Fetching multiple commodity prices...\n")
for commodity in commodities:
    ticker = COMMODITY_TICKERS[commodity]
    print(f"Fetching {commodity} ({ticker})...")

    data = fetcher.fetch_yahoo_data(
        ticker=ticker,
        start_date='2020-01-01',
        end_date='2023-12-31'
    )

    multi_commodity_data[commodity] = data['Close']

# Combine into single DataFrame
commodity_prices = pd.DataFrame(multi_commodity_data)
print(f"\nCombined data shape: {commodity_prices.shape}")
commodity_prices.head()
```

```
Fetching multiple commodity prices...

Fetching crude_oil (CL=F)...
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[9], line 10
      7     ticker = COMMODITY_TICKERS[commodity]
      8     print(f"Fetching {commodity} ({ticker})...")
---> 10     data = fetcher.fetch_yahoo_data(
     11         ticker=ticker,
     12         start_date='2020-01-01',
     13         end_date='2023-12-31'
     14     )
     16     multi_commodity_data[commodity] = data['Close']
     18 # Combine into single DataFrame

AttributeError: 'CommodityDataFetcher' object has no attribute 'fetch_yahoo_d
```

In [10]:
```
# Normalize prices to 100 at start date for comparison
normalized_prices = (commodity_prices / commodity_prices.iloc[0] * 100)

fig, ax = plt.subplots(figsize=(14, 6))

for col in normalized_prices.columns:
    normalized_prices[col].plot(ax=ax, linewidth=2, label=col.replace('_'

ax.axhline(y=100, color='black', linestyle='--', alpha=0.5)
ax.set_ylabel('Normalized Price (Base = 100)')
ax.set_xlabel('Date')
ax.set_title('Relative Performance of Different Commodities')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 2
      1 # Normalize prices to 100 at start date for comparison
----> 2 normalized_prices = (commodity_prices / commodity_prices.iloc[0] * 10
      4 fig, ax = plt.subplots(figsize=(14, 6))
      6 for col in normalized_prices.columns:

NameError: name 'commodity_prices' is not defined
```

2.4 CommodityDataFetcher Class Demonstration

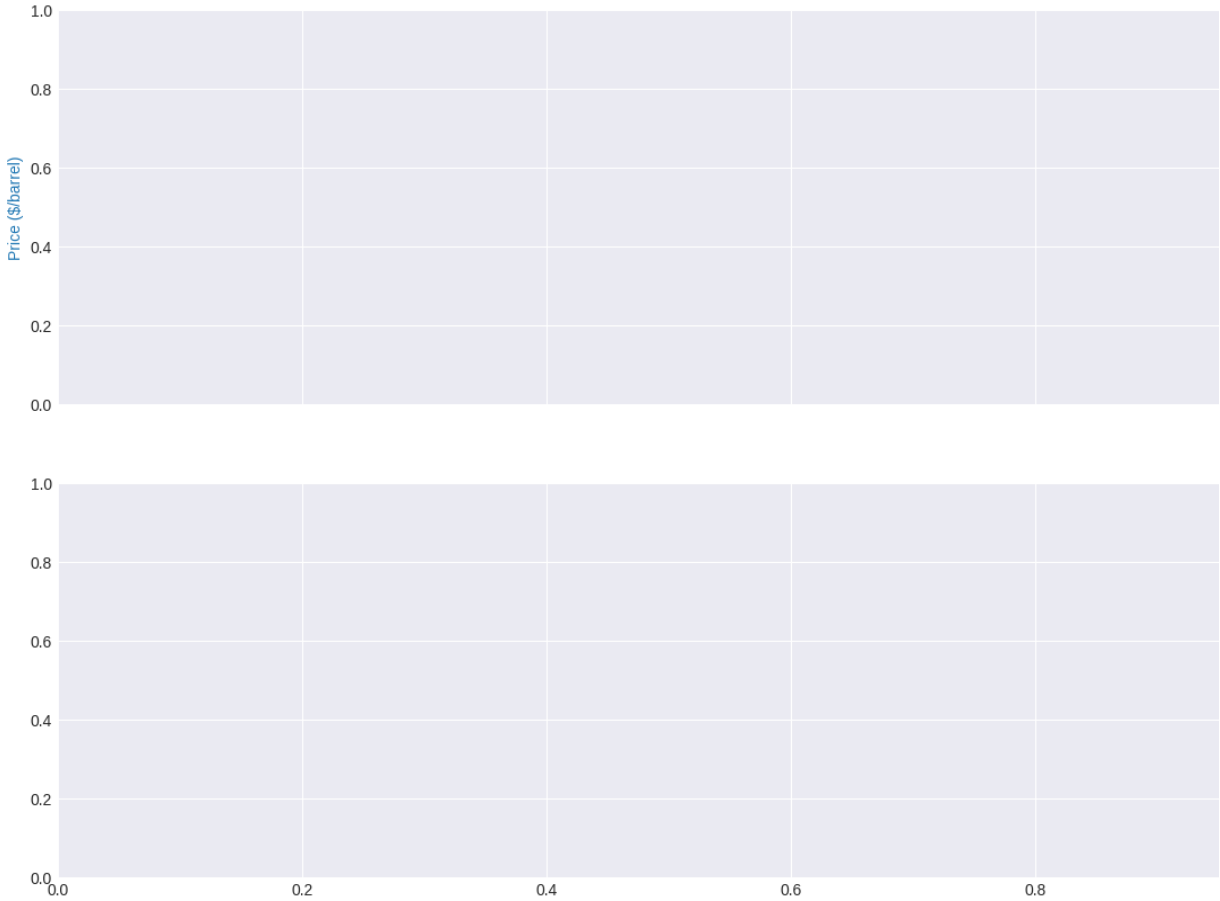
In [11]:
```
# Use the high-level fetch method to get comprehensive dataset
comprehensive_data = fetcher.fetch_commodity_data(
    commodity='crude_oil',
    start_date='2020-01-01',
    end_date='2023-12-31',
    include_fundamentals=True,
    include_economic=True
)

print(f"Comprehensive dataset shape: {comprehensive_data.shape}")
print(f"\nColumns: {list(comprehensive_data.columns)}")
comprehensive_data.head()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[11], line 2
      1 # Use the high-level fetch method to get comprehensive dataset
----> 2 comprehensive_data = fetcher.fetch_commodity_data(
      3     commodity='crude_oil',
      4     start_date='2020-01-01',
      5     end_date='2023-12-31',
      6     include_fundamentals=True,
      7     include_economic=True
      8 )
     10 print(f"Comprehensive dataset shape: {comprehensive_data.shape}")
     11 print(f"\nColumns: {list(comprehensive_data.columns)}")

AttributeError: 'CommodityDataFetcher' object has no attribute
'fetch_commodity_data'
```

In [12]:

```
# Visualize relationship between price and inventory
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)

# Price
ax1 = axes[0]
color = 'tab:blue'
ax1.set_ylabel('Price ($/barrel)', color=color)
ax1.plot(comprehensive_data.index, comprehensive_data['price'], color=col
ax1.tick_params(axis='y', labelcolor=color)
ax1.grid(True, alpha=0.3)
ax1.set_title('Price vs Inventory - Crude Oil')

# Inventory on secondary axis
ax2 = ax1.twinx()
color = 'tab:orange'
ax2.set_ylabel('Inventory (million barrels)', color=color)
ax2.plot(comprehensive_data.index, comprehensive_data['inventory'], color
ax2.tick_params(axis='y', labelcolor=color)

# Scatter plot for correlation analysis
axes[1].scatter(
    comprehensive_data['inventory'],
    comprehensive_data['price'],
    alpha=0.5,
    s=20
)

# Add trend line
z = np.polyfit(
    comprehensive_data['inventory'].dropna(),
    comprehensive_data['price'].dropna(),
    1
)
p = np.poly1d(z)
axes[1].plot(
    comprehensive_data['inventory'].sort_values(),
    p(comprehensive_data['inventory'].sort_values()),
    "r--",
    linewidth=2,
    label=f'y={z[0]:.2f}x+{z[1]:.2f}'
)

# Calculate correlation
corr = comprehensive_data[['price', 'inventory']].corr().iloc[0, 1]
axes[1].set_xlabel('Inventory (million barrels)')
axes[1].set_ylabel('Price ($/barrel)')
axes[1].set_title(f'Price-Inventory Relationship (Correlation: {corr:.3f}
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[12], line 8
      6 color = 'tab:blue'
      7 ax1.set_ylabel('Price ($/barrel)', color=color)
----> 8 ax1.plot(comprehensive_data.index, comprehensive_data['price'],
color=color, linewidth=1.5)
      9 ax1.tick_params(axis='y', labelcolor=color)
     10 ax1.grid(True, alpha=0.3)

NameError: name 'comprehensive_data' is not defined
```



Part 3 : Data Quality Assessment

3.1 Completeness Checks

In [13]:
```
def assess_completeness(df, threshold=0.95):
    """
    Assess data completeness for each column.

    Parameters
    ----------
    df : pd.DataFrame
        Dataset to assess
    threshold : float
        Minimum acceptable completeness (0-1)

    Returns
    -------
    pd.DataFrame
        Completeness statistics
    """
```

```
    results = []

    for col in df.columns:
        total = len(df)
        non_null = df[col].notna().sum()
        completeness = non_null / total

        results.append({
            'column': col,
            'total_rows': total,
            'non_null': non_null,
            'missing': total - non_null,
            'completeness': completeness,
            'meets_threshold': completeness >= threshold
        })

    return pd.DataFrame(results)

completeness = assess_completeness(comprehensive_data, threshold=0.95)
print("Data Completeness Assessment:\n")
print(completeness.to_string(index=False))
```

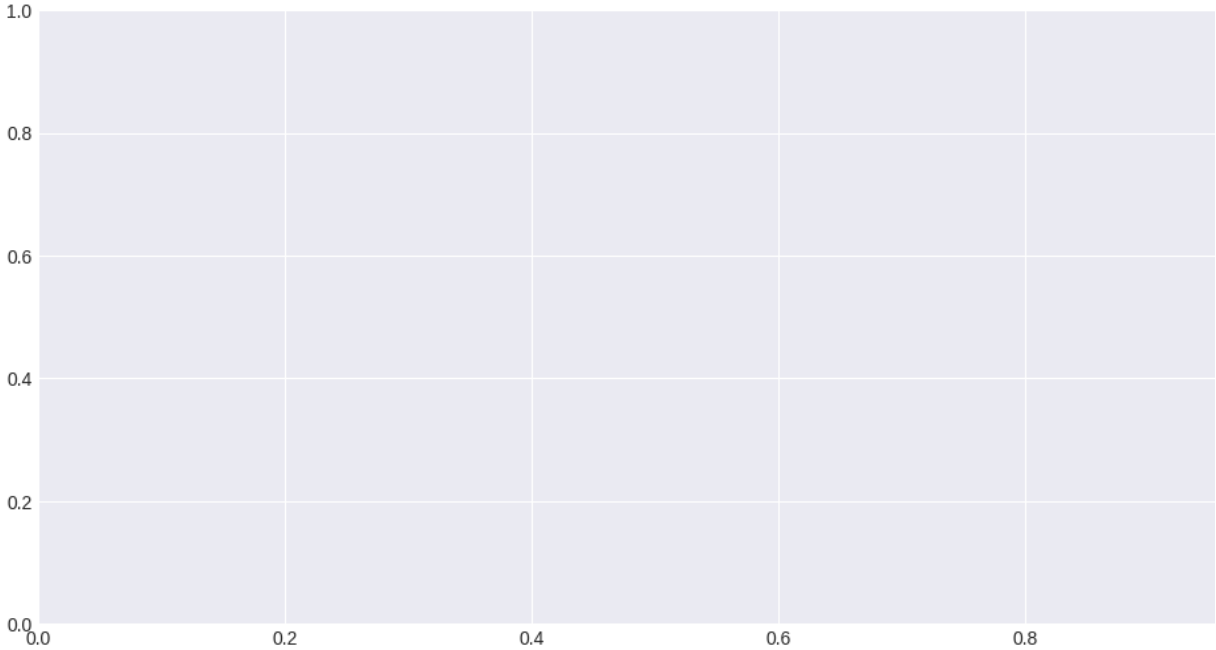
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[13], line 35
     24         results.append({
     25             'column': col,
     26             'total_rows': total,
   (...)     30             'meets_threshold': completeness >= threshold
     31         })
     33     return pd.DataFrame(results)
---> 35 completeness = assess_completeness(comprehensive_data, threshold=0.95
     36 print("Data Completeness Assessment:\n")
     37 print(completeness.to_string(index=False))

NameError: name 'comprehensive_data' is not defined
```

In [14]:

```
# Visualize completeness
fig, ax = plt.subplots(figsize=(12, 6))

colors = ['green' if x else 'red' for x in completeness['meets_threshold'
completeness.plot(
    x='column',
    y='completeness',
    kind='bar',
    ax=ax,
    color=colors,
    legend=False
)

ax.axhline(y=0.95, color='orange', linestyle='--', linewidth=2, label='95
ax.set_ylabel('Completeness Ratio')
ax.set_xlabel('Column')
ax.set_title('Data Completeness by Column')
ax.set_ylim(0, 1.05)
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.xticks(rotation=45, ha='right')
```

```
plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[14], line 4
      1 # Visualize completeness
      2 fig, ax = plt.subplots(figsize=(12, 6))
----> 4 colors = ['green' if x else 'red' for x in completeness['meets_thresh
      5 completeness.plot(
      6     x='column',
      7     y='completeness',
   (...)     11     legend=False
     12 )
     14 ax.axhline(y=0.95, color='orange', linestyle='--', linewidth=2, label
threshold')

NameError: name 'completeness' is not defined
```



3.2 Outlier Detection (Temporally Valid)

In [15]:
```
def detect_outliers_expanding(series, threshold=3.0, min_periods=30):
    """
    Detect outliers using expanding window (temporally valid).

    At each point, only uses data available up to that point.
    """
    # Calculate expanding z-scores
    expanding_mean = series.expanding(min_periods=min_periods).mean()
    expanding_std = series.expanding(min_periods=min_periods).std()

    z_scores = (series - expanding_mean) / expanding_std

    # Flag outliers
    outliers = z_scores.abs() > threshold

    return outliers, z_scores

# Detect outliers in price returns
returns = comprehensive_data['price'].pct_change() * 100
```

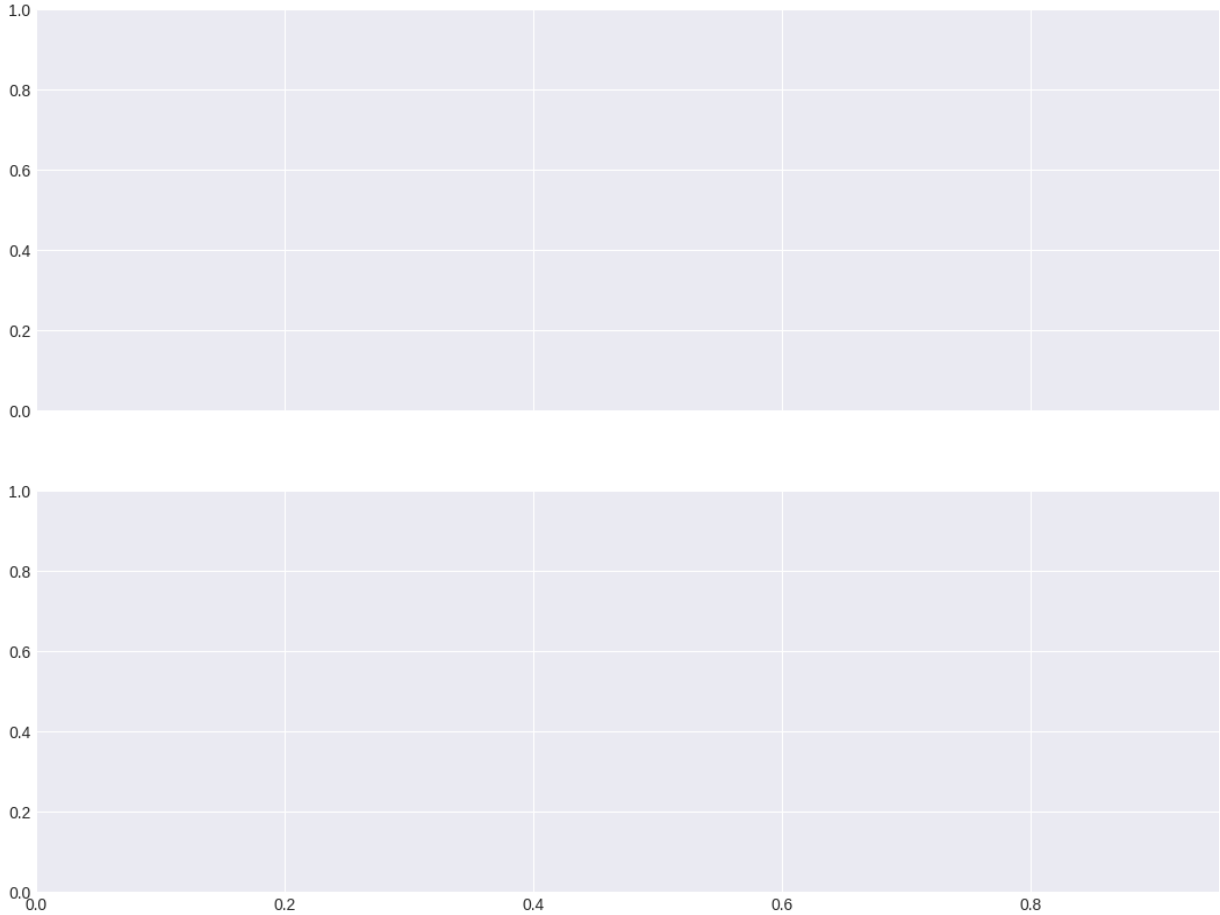
```
outliers, z_scores = detect_outliers_expanding(returns, threshold=3.0)

print(f"Detected {outliers.sum()} outliers ({outliers.sum()/len(returns)*
print(f"\nOutlier dates:")
print(returns[outliers].sort_values(ascending=False).head(10))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[15], line 19
     16     return outliers, z_scores
     18 # Detect outliers in price returns
---> 19 returns = comprehensive_data['price'].pct_change() * 100
     20 outliers, z_scores = detect_outliers_expanding(returns, threshold=3.0
     22 print(f"Detected {outliers.sum()} outliers ({outliers.sum()/len(retur
0:.2f}% of data)")

NameError: name 'comprehensive_data' is not defined
```

In [16]:

```
# Visualize outliers
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)

# Returns with outliers highlighted
returns.plot(ax=axes[0], linewidth=1, alpha=0.7, color='blue')
returns[outliers].plot(
    ax=axes[0],
    marker='o',
    linestyle='',
    markersize=8,
    color='red',
    label='Outliers'
)
axes[0].set_ylabel('Daily Return (%)')
axes[0].set_title('Price Returns with Outliers Highlighted')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
axes[0].axhline(y=0, color='black', linestyle='-', alpha=0.3)

# Z-scores
z_scores.plot(ax=axes[1], linewidth=1, color='purple')
axes[1].axhline(y=3, color='red', linestyle='--', alpha=0.5, label='±3σ t
axes[1].axhline(y=-3, color='red', linestyle='--', alpha=0.5)
axes[1].axhline(y=0, color='black', linestyle='-', alpha=0.3)
axes[1].fill_between(z_scores.index, -3, 3, alpha=0.1, color='green')
axes[1].set_ylabel('Z-Score')
axes[1].set_xlabel('Date')
axes[1].set_title('Expanding Window Z-Scores')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[16], line 5
      2 fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)
      4 # Returns with outliers highlighted
----> 5 returns.plot(ax=axes[0], linewidth=1, alpha=0.7, color='blue')
      6 returns[outliers].plot(
      7     ax=axes[0],
      8     marker='o',
   (...)     12     label='Outliers'
     13 )
     14 axes[0].set_ylabel('Daily Return (%)')

NameError: name 'returns' is not defined
```



3.3 Cross-Source Validation

In [17]:
```
# Compare WTI price from Yahoo Finance vs FRED
yahoo_wti = fetcher.fetch_yahoo_data(
    ticker='CL=F',
    start_date='2020-01-01',
    end_date='2023-12-31'
)['Close']

fred_wti = fetcher.fetch_fred_data(
    series_id='DCOILWTICO',  # WTI spot price
    start_date='2020-01-01',
    end_date='2023-12-31'
)

# Align data
comparison = pd.DataFrame({
```

```
    'yahoo_futures': yahoo_wti,
    'fred_spot': fred_wti
}).dropna()

# Calculate difference
comparison['difference'] = comparison['yahoo_futures'] - comparison['fred
comparison['pct_difference'] = (comparison['difference'] / comparison['fr

print("Cross-Source Validation Statistics:\n")
print(f"Correlation: {comparison['yahoo_futures'].corr(comparison['fred_s
print(f"Mean difference: ${comparison['difference'].mean():.2f}")
print(f"Mean % difference: {comparison['pct_difference'].mean():.2f}%")
print(f"Std of difference: ${comparison['difference'].std():.2f}")
```

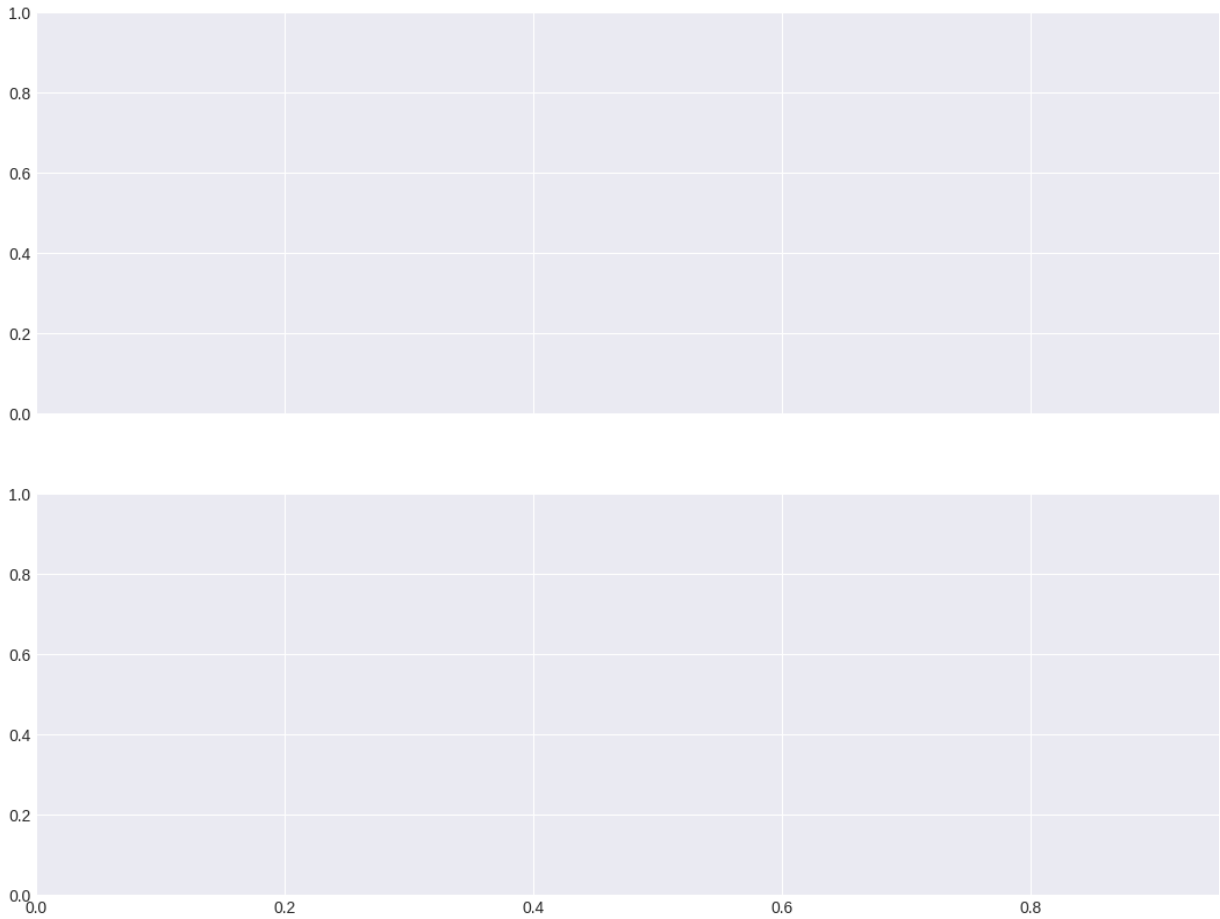
```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[17], line 2
      1 # Compare WTI price from Yahoo Finance vs FRED
----> 2 yahoo_wti = fetcher.fetch_yahoo_data(
      3     ticker='CL=F',
      4     start_date='2020-01-01',
      5     end_date='2023-12-31'
      6 )['Close']
      8 fred_wti = fetcher.fetch_fred_data(
      9     series_id='DCOILWTICO',  # WTI spot price
     10     start_date='2020-01-01',
     11     end_date='2023-12-31'
     12 )
     14 # Align data

AttributeError: 'CommodityDataFetcher' object has no attribute 'fetch_yahoo_d
```

In [18]:

```
# Visualize cross-source comparison
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)

# Both series
comparison[['yahoo_futures', 'fred_spot']].plot(ax=axes[0], linewidth=1.5
axes[0].set_ylabel('Price ($/barrel)')
axes[0].set_title('WTI Price: Yahoo Finance Futures vs FRED Spot')
axes[0].legend(['Yahoo Futures', 'FRED Spot'])
axes[0].grid(True, alpha=0.3)

# Difference (futures premium)
comparison['difference'].plot(ax=axes[1], linewidth=1.5, color='green')
axes[1].axhline(y=0, color='black', linestyle='--', alpha=0.5)
axes[1].fill_between(
    comparison.index,
    comparison['difference'],
    0,
    where=(comparison['difference'] > 0),
    alpha=0.3,
    color='green',
    label='Contango'
)
axes[1].fill_between(
    comparison.index,
    comparison['difference'],
    0,
    where=(comparison['difference'] < 0),
    alpha=0.3,
```

```
    color='red',
    label='Backwardation'
)
axes[1].set_ylabel('Futures Premium ($/barrel)')
axes[1].set_xlabel('Date')
axes[1].set_title('Futures-Spot Spread (Convenience Yield Signal)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[18], line 5
      2 fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)
      4 # Both series
----> 5 comparison[[                ,             ]].plot(ax=axes[0], linewidth
alpha=0.8)
      6 axes[0].set_ylabel('Price ($/barrel)')
      7 axes[0].set_title('WTI Price: Yahoo Finance Futures vs FRED Spot')

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/frame.py:4119, in DataFrame.__getitem__(
key)
   4117     if is_iterator(key):
   4118         key = list(key)
-> 4119     indexer = self.columns._get_indexer_strict(key,           )[1]
   4121 # take() does not accept boolean indexers
   4122 if getattr(indexer, "dtype", None) == bool:

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/indexes/base.py:6212, in Index._get_inde
rict(self, key, axis_name)
   6209 else:
   6210     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 6212 self._raise_if_missing(keyarr, indexer, axis_name)
   6214 keyarr = self.take(indexer)
   6215 if isinstance(key, Index):
   6216     # GH 42790 - Preserve name from an Index

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/indexes/base.py:6261, in Index._raise_if
ng(self, key, indexer, axis_name)
   6259 if nmissing:
   6260     if nmissing == len(indexer):
-> 6261         raise KeyError(f"None of [{key}] are in the [{axis_name}]")
   6263     not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].uni
   6264     raise KeyError(f"{not_found} not in index")

KeyError: "None of [Index(['yahoo_futures', 'fred_spot'], dtype='object')] ar
the [columns]"
```



Part 4 : Handling Data Issues

4.1 Missing Data Imputation (Temporally Valid)

In [19]:
```
def impute_missing_temporal(series, method='forward_fill', max_gap=5):
    """
    Impute missing values using temporally valid methods.

    Parameters
    ----------
    series : pd.Series
        Series with missing values
    method : str
        Imputation method:
        - 'forward_fill': Use last known value (most conservative)
        - 'linear': Linear interpolation (assumes smooth evolution)
        - 'expanding_mean': Use expanding window mean
    max_gap : int
        Maximum gap size to fill (larger gaps left as NaN)

    Returns
    -------
    pd.Series
        Imputed series
    """
    result = series.copy()

    if method == 'forward_fill':
        result = result.ffill(limit=max_gap)

    elif method == 'linear':
```

```
        # Only interpolate small gaps
        result = result.interpolate(method='linear', limit=max_gap)

    elif method == 'expanding_mean':
        # Fill with expanding mean up to that point
        expanding_mean = result.expanding(min_periods=1).mean()

        # Only fill small gaps
        mask = result.isna()
        gap_size = mask.groupby((~mask).cumsum()).cumsum()
        fill_mask = mask & (gap_size <= max_gap)

        result[fill_mask] = expanding_mean[fill_mask]

    return result

# Create sample series with missing data
sample = comprehensive_data['production'].copy()
sample_with_gaps = sample.copy()

# Introduce random gaps
np.random.seed(42)
gap_indices = np.random.choice(sample.index[50:], size=30, replace=False)
sample_with_gaps.loc[gap_indices] = np.nan

# Test different imputation methods
imputed_ff = impute_missing_temporal(sample_with_gaps, method='forward_fi
imputed_linear = impute_missing_temporal(sample_with_gaps, method='linear
imputed_mean = impute_missing_temporal(sample_with_gaps, method='expandin

print(f"Original missing: {sample_with_gaps.isna().sum()}")
print(f"After forward fill: {imputed_ff.isna().sum()}")
print(f"After linear: {imputed_linear.isna().sum()}")
print(f"After expanding mean: {imputed_mean.isna().sum()}")
```

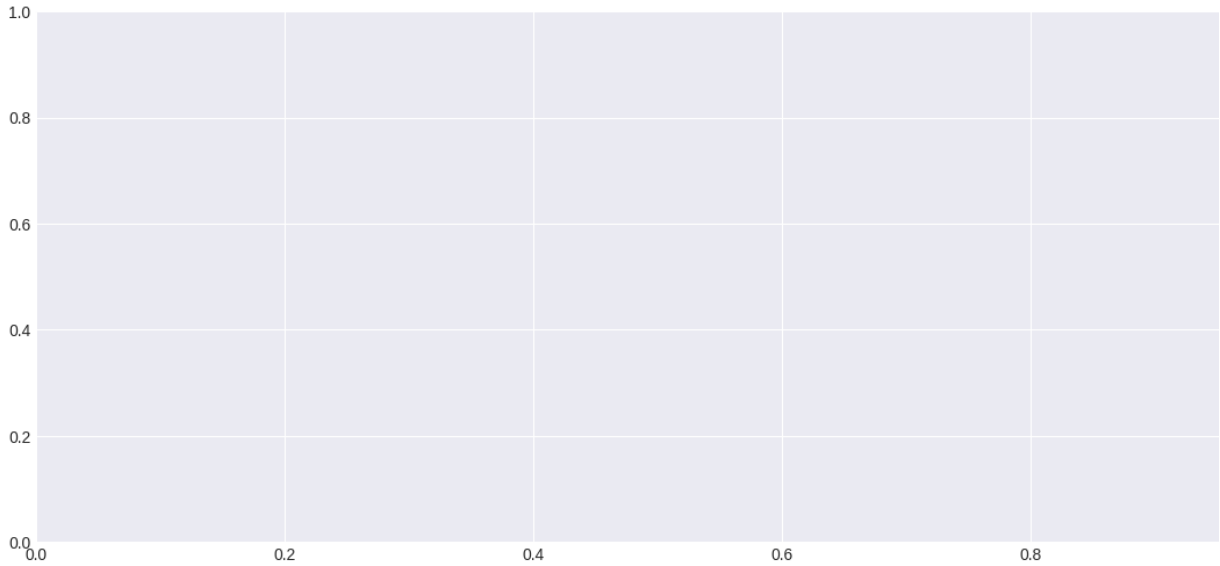
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[19], line 45
     42     return result
     44 # Create sample series with missing data
---> 45 sample = comprehensive_data['production'].copy()
     46 sample_with_gaps = sample.copy()
     48 # Introduce random gaps

NameError: name 'comprehensive_data' is not defined
```

In [20]:

```
# Visualize imputation methods
fig, ax = plt.subplots(figsize=(14, 6))

# Plot a zoomed-in section
zoom_range = slice('2022-01-01', '2022-06-30')

sample.loc[zoom_range].plot(ax=ax, linewidth=2, label='Original', color='
sample_with_gaps.loc[zoom_range].plot(
    ax=ax, marker='o', markersize=4, linestyle='', label='With Gaps', col
)
imputed_ff.loc[zoom_range].plot(
    ax=ax, linewidth=1.5, linestyle='--', label='Forward Fill', alpha=0.7
)
imputed_linear.loc[zoom_range].plot(
```

```
    ax=ax, linewidth=1.5, linestyle='--', label='Linear', alpha=0.7
)
imputed_mean.loc[zoom_range].plot(
    ax=ax, linewidth=1.5, linestyle='--', label='Expanding Mean', alpha=0
)

ax.set_ylabel('Production')
ax.set_xlabel('Date')
ax.set_title('Comparison of Temporal Imputation Methods')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[20], line 7
      4 # Plot a zoomed-in section
      5 zoom_range = slice('2022-01-01', '2022-06-30')
----> 7 sample.loc[zoom_range].plot(ax=ax, linewidth=2, label='Original', col
ack', alpha=0.7)
      8 sample_with_gaps.loc[zoom_range].plot(
      9     ax=ax, marker='o', markersize=4, linestyle='', label='With Gaps',
color='red'
     10 )
     11 imputed_ff.loc[zoom_range].plot(
     12     ax=ax, linewidth=1.5, linestyle='--', label='Forward Fill', alpha
     13 )

NameError: name 'sample' is not defined
```



4.2 Outlier Treatment

In [21]:
```
def treat_outliers(series, method='winsorize', threshold=3.0, quantile=0.
    """
    Treat outliers using various methods.

    Parameters
    ----------
    series : pd.Series
        Series with potential outliers
    method : str
```

```
        Treatment method:
        - 'winsorize': Cap at threshold
        - 'remove': Set to NaN
        - 'median': Replace with expanding median
    threshold : float
        Z-score threshold for outlier detection
    quantile : float
        Quantile for winsorization

    Returns
    -------
    pd.Series
        Treated series
    """
    result = series.copy()

    # Detect outliers using expanding window
    expanding_mean = series.expanding(min_periods=30).mean()
    expanding_std = series.expanding(min_periods=30).std()
    z_scores = (series - expanding_mean) / expanding_std
    outliers = z_scores.abs() > threshold

    if method == 'winsorize':
        # Cap at expanding quantiles
        upper = series.expanding(min_periods=30).quantile(1 - quantile)
        lower = series.expanding(min_periods=30).quantile(quantile)
        result = result.clip(lower=lower, upper=upper)

    elif method == 'remove':
        result[outliers] = np.nan

    elif method == 'median':
        expanding_median = series.expanding(min_periods=30).median()
        result[outliers] = expanding_median[outliers]

    return result

# Treat outliers in returns
returns_winsorized = treat_outliers(returns, method='winsorize', threshol
returns_removed = treat_outliers(returns, method='remove', threshold=3.0)
returns_median = treat_outliers(returns, method='median', threshold=3.0)

print("Outlier Treatment Comparison:\n")
print(f"Original - Mean: {returns.mean():.3f}%, Std: {returns.std():.3f}%
print(f"Winsorized - Mean: {returns_winsorized.mean():.3f}%, Std: {return
print(f"Removed - Mean: {returns_removed.mean():.3f}%, Std: {returns_remo
print(f"Median - Mean: {returns_median.mean():.3f}%, Std: {returns_median
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[21], line 48
     45     return result
     47 # Treat outliers in returns
---> 48 returns_winsorized = treat_outliers(returns, method='winsorize',
threshold=3.0)
     49 returns_removed = treat_outliers(returns, method='remove', threshold=
     50 returns_median = treat_outliers(returns, method='median', threshold=3

NameError: name 'returns' is not defined
```

4.3 Aligning Frequencies

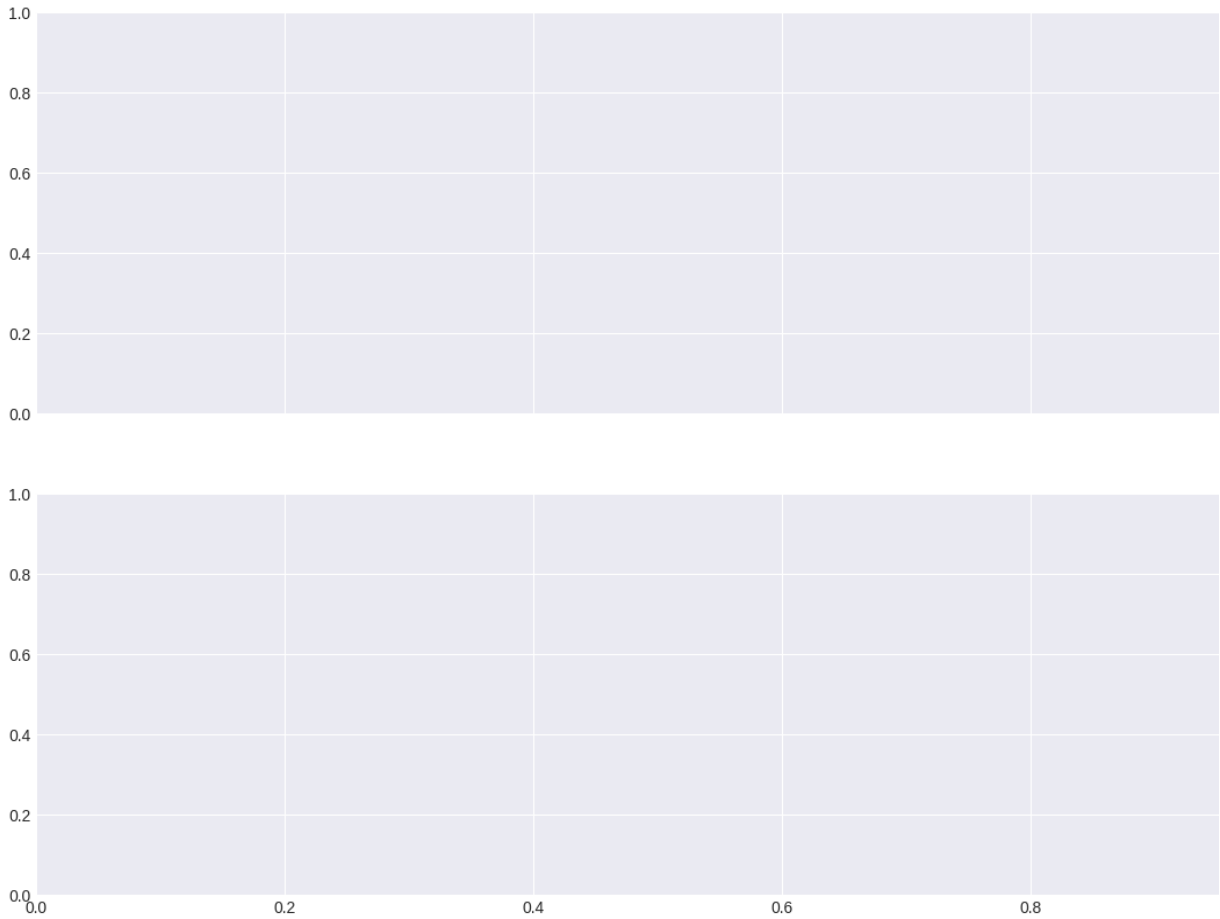
In [22]:
```
def align_frequencies(df, target_freq='D', method='forward_fill'):
    """
    Align mixed-frequency data to a common frequency.

    Parameters
    ----------
    df : pd.DataFrame
        DataFrame with potentially mixed frequencies
    target_freq : str
        Target frequency ('D' for daily, 'W' for weekly, etc.)
    method : str
        How to handle missing values after resampling

    Returns
    -------
    pd.DataFrame
        Aligned DataFrame
    """
    # Resample to target frequency
    result = df.resample(target_freq).first()  # Take first value in peri

    # Fill missing values
    if method == 'forward_fill':
        result = result.ffill()
    elif method == 'interpolate':
        result = result.interpolate(method='linear')

    return result

# Example: Align weekly inventory to daily price data
aligned_data = align_frequencies(
    comprehensive_data[['price', 'inventory']],
    target_freq='D',
    method='forward_fill'
)

print(f"Original data points: {len(comprehensive_data)}")
print(f"Aligned data points: {len(aligned_data)}")
print(f"\nMissing values after alignment:")
print(aligned_data.isna().sum())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[22], line 32
     28     return result
     30 # Example: Align weekly inventory to daily price data
     31 aligned_data = align_frequencies(
---> 32     comprehensive_data[['price', 'inventory']],
     33     target_freq='D',
     34     method='forward_fill'
     35 )
     37 print(f"Original data points: {len(comprehensive_data)}")
     38 print(f"Aligned data points: {len(aligned_data)}")

NameError: name 'comprehensive_data' is not defined
```

In [23]:
```
# Visualize frequency alignment
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)
```

```
# Original (mixed frequency)
zoom = slice('2022-01-01', '2022-03-31')
comprehensive_data.loc[zoom, 'inventory'].plot(
    ax=axes[0], marker='o', linestyle='-', linewidth=1.5, markersize=6, l
)
axes[0].set_ylabel('Inventory (million barrels)')
axes[0].set_title('Original Weekly Inventory Data')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Aligned (daily)
aligned_data.loc[zoom, 'inventory'].plot(
    ax=axes[1], marker='o', linestyle='-', linewidth=1, markersize=3, lab
)
axes[1].set_ylabel('Inventory (million barrels)')
axes[1].set_xlabel('Date')
axes[1].set_title('Aligned Daily Inventory Data')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[23], line 6
      4 # Original (mixed frequency)
      5 zoom = slice('2022-01-01', '2022-03-31')
----> 6 comprehensive_data.loc[zoom, 'inventory'].plot(
      7     ax=axes[0], marker='o', linestyle='-', linewidth=1.5, markersize=
label='Original (Weekly)'
      8 )
      9 axes[0].set_ylabel('Inventory (million barrels)')
     10 axes[0].set_title('Original Weekly Inventory Data')

NameError: name 'comprehensive_data' is not defined
```



4.4 Building a Robust Data Pipeline

In [24]:
```
def build_data_pipeline(commodity, start_date, end_date,
                        quality_threshold=0.9, outlier_threshold=3.0):
    """
    Build a complete data pipeline with quality checks.

    Steps:
    1. Fetch data from multiple sources
    2. Assess quality
    3. Handle missing values
    4. Treat outliers
    5. Align frequencies
    6. Create point-in-time structure
    """
    print("=" * 60)
    print(f"Building data pipeline for {commodity}")
    print(f"Period: {start_date} to {end_date}")
    print("=" * 60)

    # Step 1: Fetch data
    print("\n1. Fetching data...")
    fetcher = CommodityDataFetcher(cache_dir='../../data/cache')
    data = fetcher.fetch_commodity_data(
        commodity=commodity,
        start_date=start_date,
        end_date=end_date,
        include_fundamentals=True,
        include_economic=True
    )
    print(f"   Fetched {len(data)} rows with {len(data.columns)} columns"
```

```
    # Step 2: Quality assessment
    print("\n2. Assessing quality...")
    completeness = assess_completeness(data, threshold=quality_threshold)
    failing_cols = completeness[~completeness['meets_threshold']]['column

    if failing_cols:
        print(f"   WARNING: {len(failing_cols)} columns below quality thr
    else:
        print("   All columns meet quality threshold")

    # Step 3: Handle missing values
    print("\n3. Handling missing values...")
    for col in data.columns:
        if data[col].isna().any():
            data[col] = impute_missing_temporal(data[col], method='forwar
    print(f"   Remaining missing values: {data.isna().sum().sum()}")

    # Step 4: Treat outliers
    print("\n4. Treating outliers...")
    for col in data.select_dtypes(include=[np.number]).columns:
        returns_col = data[col].pct_change()
        outliers, _ = detect_outliers_expanding(returns_col, threshold=ou
        n_outliers = outliers.sum()
        if n_outliers > 0:
            print(f"   {col}: {n_outliers} outliers detected and winsoriz
            data[col] = treat_outliers(data[col], method='winsorize', thr

    # Step 5: Align frequencies
    print("\n5. Aligning frequencies...")
    data = align_frequencies(data, target_freq='D', method='forward_fill'
    print(f"   Aligned to daily frequency: {len(data)} rows")

    # Step 6: Create point-in-time structure
    print("\n6. Creating point-in-time structure...")
    pit_data = PointInTimeDataFrame(data)
    print("   Point-in-time structure created")

    print("\n" + "=" * 60)
    print("Pipeline complete!")
    print("=" * 60)

    return pit_data, completeness

# Build pipeline for crude oil
clean_data, quality_report = build_data_pipeline(
    commodity='crude_oil',
    start_date='2020-01-01',
    end_date='2023-12-31',
    quality_threshold=0.9,
    outlier_threshold=3.0
)
```

```
============================================================
Building data pipeline for crude_oil
Period: 2020-01-01 to 2023-12-31
============================================================

1. Fetching data...
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[24], line 75
     72     return pit_data, completeness
     74 # Build pipeline for crude oil
---> 75 clean_data, quality_report = build_data_pipeline(
     76     commodity=           ,
     77     start_date=             ,
     78     end_date=            ,
     79     quality_threshold=0.9,
     80     outlier_threshold=3.0
     81 )

Cell In[24], line 22, in build_data_pipeline(commodity, start_date, end_date,
quality_threshold, outlier_threshold)
     20 print("\n1. Fetching data...")
     21 fetcher = CommodityDataFetcher(cache_dir='../../data/cache')
---> 22 data = fetcher.fetch_commodity_data(
     23     commodity=commodity,
     24     start_date=start_date,
     25     end_date=end_date,
     26     include_fundamentals=True,
     27     include_economic=True
     28 )
     29 print(f"   Fetched {len(data)} rows with {len(data.columns)} columns"
     31 # Step 2: Quality assessment

AttributeError: 'CommodityDataFetcher' object has no attribute
'fetch_commodity_data'
```

In [25]:

```
# Display final data quality
print("Final Data Quality Report:\n")
print(quality_report[['column', 'completeness', 'meets_threshold']].to_st

print(f"\nFinal dataset shape: {clean_data.shape}")
print(f"Date range: {clean_data.index.min().date()} to {clean_data.index.
clean_data.head()
```

```
Final Data Quality Report:
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[25], line 3
      1 # Display final data quality
      2 print("Final Data Quality Report:\n")
----> 3 print(quality_report[['column', 'completeness', 'meets_threshold']].t
ng(index=False))
      5 print(f"\nFinal dataset shape: {clean_data.shape}")
      6 print(f"Date range: {clean_data.index.min().date()} to {clean_data.in
x().date()}")

NameError: name 'quality_report' is not defined
```

Summary and Key Takeaways

What We've Learned

1. **Data Sources**:
   - Free sources (Yahoo, FRED, EIA) provide substantial data for research
   - Premium sources offer more depth and reliability
   - Different sources have different strengths (prices vs fundamentals)
2. **API Integration**:
   - yfinance for futures prices
   - FRED for economic data and some commodity fundamentals
   - CommodityDataFetcher class for unified access
3. **Quality Assessment**:
   - Completeness: % of non-null values
   - Outlier detection: Using expanding windows to avoid look-ahead bias
   - Cross-source validation: Compare similar data from different sources
4. **Data Issues**:
   - Missing data: Use forward fill or interpolation (temporally valid)
   - Outliers: Winsorize or remove using expanding statistics
   - Mixed frequencies: Align to common frequency with forward fill

Critical Principles

1. **Always use expanding windows** for statistics and outlier detection
2. **Never use future data** to impute or validate past data
3. **Document data provenance** - know where each data point came from
4. **Validate across sources** when possible
5. **Build automated pipelines** to ensure consistency

Next Steps

In the next module, we'll use this clean, validated data to build fair value models:

- Inventory-based models
- Cost-of-carry models
- Supply-demand balance models
- Ensemble approaches

Exercises

1. **Multi-Source Pipeline**: Build a data pipeline that fetches the same commodity from sources and creates a consensus estimate.

2. **Quality Monitoring**: Create a function that monitors data quality over time and alerts when degrades below thresholds.

3. **Advanced Outlier Detection**: Implement a more sophisticated outlier detector that conside

    - Market regime (higher threshold in volatile periods)
    - Day-of-week effects
    - Seasonality

4. **Revision Tracking**: Build a system to track and store multiple vintages of the same data se

5. **Custom Imputation**: Create a custom imputation method that uses related commodities to missing values (e.g., use Brent to estimate missing WTI values).